

Backtracking — Complete Guide

Concept, recognition signals, templates, and complexity — Java edition

1. What is Backtracking?

Backtracking is a technique for solving problems by building a solution incrementally, one choice at a time, and abandoning ("backtracking") a partial solution as soon as you determine it can't possibly lead to a valid complete solution. It's essentially **DFS on a decision tree** where you prune branches that can't work.

Think of it as: **try** → **recurse** → **undo (if it didn't work)** → **try next option**.

2. When & Why is it Used?

Used when you need to explore **all (or some) possible configurations** to find solutions that satisfy constraints, and there's no formula or greedy shortcut — you must search the space of possibilities.

Why:

- Brute force (generate everything, then check) wastes time building invalid states fully before rejecting them.
- Backtracking prunes early — checks validity mid-construction and abandons a branch the moment it's doomed, instead of completing it first.

3. Where is it Used (Real World / Interview Problems)?

- Combinatorial generation: permutations, subsets, combinations
- Constraint satisfaction: N-Queens, Sudoku, graph coloring
- Path finding: word search, maze solving, Hamiltonian path
- Partitioning: palindrome partitioning, subset-sum partitioning
- Parsing/generation: generating valid parentheses, IP address restoration
- Puzzle solving: crosswords, cryptarithmic

4. How Do You Identify a Backtracking Problem? (The 2-Minute Signal)

Phrase in Problem	Signal
"Find all possible..." / "Generate all..."	Strong signal
"Return all combinations/permutations/subsets"	Strong signal
"Count the number of ways to..."	Backtracking (or DP if overlapping subproblems)

"Is it possible to place/arrange... such that [constraints]"	Backtracking
Board/grid/positions with placement rules	N-Queens/Sudoku-style
"Partition into..."	Backtracking
Words like valid, distinct, arrangement, configuration	Backtracking

Rule of thumb: the problem has a **sequence of decisions**, each decision has **multiple choices**, and **some choices become invalid based on earlier choices** — so you need to explore, check validity, and undo. If you catch yourself thinking "I'd try this, and if it doesn't work I'd undo it and try the next thing" — that's backtracking.

5. Can You Identify it From Constraints Alone?

Yes — constraints are often the tell:

- **Small input size** ($n \leq 10-20$ for permutations/subsets, $n \leq 9$ for Sudoku-like grids) → screams backtracking/exponential search, since polynomial solutions aren't being demanded.
- If n were large (10^5+), backtracking is infeasible — small n is the problem-setter's way of telling you exponential/backtracking is intended.
- "Return all solutions" (vs. "return the count" or "return true/false") → usually means you must enumerate, i.e., backtrack. Counting/boolean can sometimes be optimized with DP instead.

6. Types of Backtracking Problems

Type	Description	Example
Decision	Does a solution exist?	Sudoku solvable?
Optimization	Find the best solution	Often better via DP/greedy if overlapping subproblems exist
Enumeration	Find all solutions	Permutations, subsets, all N-Queens solutions

By structure:

- **Subset/choose-or-skip** (each element: include or exclude) — subsets, subset sum
- **Permutation-style** (order matters, use a visited array) — permutations
- **Grid/board placement** (row-by-row or cell-by-cell) — N-Queens, Sudoku
- **String partitioning/building** — palindrome partitioning, parentheses generation

7. Reusable Code Template (Java)

Generic skeleton:

```

class Solution {
    public List<List<Integer>> solve(int[] input) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(new ArrayList<>(), input, 0, result);
        return result;
    }

    private void backtrack(List<Integer> path, int[] input, int start,
                           List<List<Integer>> result) {
        if (isSolution(path, input)) {
            result.add(new ArrayList<>(path)); // save a COPY
            // return; // omit if you want to keep exploring past a
            // valid state (e.g. subsets)
        }

        for (int i = start; i < input.length; i++) {
            if (!isValid(input[i], path)) continue; // pruning

            path.add(input[i]); // choose
            // mark_used(input[i]); // update state if needed

            backtrack(path, input, i + 1, result); // explore

            path.remove(path.size() - 1); // unchoose (undo)
            // unmark_used(input[i]); // revert state if needed
        }
    }

    private boolean isSolution(List<Integer> path, int[] input) { return false; }
    private boolean isValid(int candidate, List<Integer> path) { return true; }
}

```

Permutations variant (visited array, order matters):

```

class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        boolean[] used = new boolean[nums.length];
        backtrack(new ArrayList<>(), nums, used, result);
        return result;
    }

    private void backtrack(List<Integer> path, int[] nums, boolean[] used,
                           List<List<Integer>> result) {
        if (path.size() == nums.length) {
            result.add(new ArrayList<>(path));
            return;
        }

        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            path.add(nums[i]);

            backtrack(path, nums, used, result);

            path.remove(path.size() - 1);
            used[i] = false;
        }
    }
}

```

Grid/board placement variant (N-Queens style):

```
class Solution {
    public int solveNQueens(int n) {
        int[] cols = new int[n]; // cols[row] = column queen is placed in
        return backtrack(0, n, cols);
    }

    private int backtrack(int row, int n, int[] cols) {
        if (row == n) return 1; // found a valid full placement

        int count = 0;
        for (int col = 0; col < n; col++) {
            if (!isValid(row, col, cols)) continue; // pruning

            cols[row] = col; // place queen
            count += backtrack(row + 1, n, cols); // explore next row
            // no explicit "undo" needed: cols[row] is overwritten
            // on the next loop iteration
        }
        return count;
    }

    private boolean isValid(int row, int col, int[] cols) {
        for (int r = 0; r < row; r++) {
            int c = cols[r];
            if (c == col || Math.abs(c - col) == row - r) return false;
        }
        return true;
    }
}
```

Key skeleton across all variants: **choose** → **explore** → **un-choose**. The "un-choose" (undo) step is what makes it backtracking, not just plain DFS.

8. Rationale / Intuition

You're walking a decision tree. At every node you have several branches (choices). Instead of exploring the entire tree blindly (brute force), you check a **validity condition as early as possible** and skip entire subtrees that can't lead anywhere — this is called **pruning**, and it's the entire reason backtracking beats brute force in practice, even though the worst-case complexity is often the same.

The "undo" step matters because you're reusing the same path/state object (a List, an array, a boolean[]) across branches instead of allocating a new one each time — this is what keeps space complexity low ($O(\text{depth})$) instead of $O(\text{depth})$ copies at every node).

9. Time & Space Complexity (Generic)

Time: Typically $O(b^d)$ where b = branching factor (choices per step) and d = depth (decision points).

- Subsets: $O(2^n)$
- Permutations: $O(n!)$
- N-Queens: roughly $O(n!)$, pruned much lower in practice

- It's exponential/factorial in the worst case — backtracking doesn't change the theoretical worst-case bound, it just avoids wasted work via pruning, so actual runtime is usually far better than the bound suggests.

Space: $O(d)$ for the recursion stack/path (depth of tree), plus $O(d)$ or $O(n)$ for extra bookkeeping (used[] array, board state, etc.). Output storage is separate — if you're collecting all solutions, that adds $O(\text{number of solutions} \times \text{solution size})$, since each solution typically gets copied into the result list (new ArrayList<>(path)).

10. Quick Decision Checklist

- Do I need to find **all/some valid ways** to arrange, choose, or place things? → lean backtracking
- Does each choice **restrict future choices** (need to check validity as I build)? → strong signal
- Is the input size small ($n \leq \sim 20$)? → confirms exponential/backtracking is intended
- Does the problem ask for **all solutions** rather than just a count or true/false? → strong signal (count/boolean might have a DP shortcut instead)
- None of the above, and there's a clear greedy/DP recurrence with overlapping subproblems? → you probably don't need backtracking, a DP solution is simpler and expected